

Riding the Hot Hand: Modeling Path Dependencies in MLB Baseball Playing Time Projections

Candidate ID: 65770

August 5, 2026

In this project, I leverage the comparative advantages of Bayesian models – automatic uncertainty quantification and the relative ease of building hierarchical modeling pipelines — to build a baseball team performance projection system. I build a regression model for player performance projection, a classification model for playing time projection, and a simulation procedure that both models feed into to produce team level performance projections. There are two main goals of my project: to build a projection system that is competitive with industry leading team performance projections and to test whether modeling the dependence of individual playing time on individual performance, which is not done in current projection systems, is a promising avenue of improvement in current models. Surprisingly, I find that while my results do not support my hypothesis on the importance of modeling this path dependence, the projection pipeline I build is competitive with other top models.

1 Introduction

1.1 Background

In recent decades, major league baseball (MLB) has become one of the most prominent data-driven sports (Mizels et al., 2022). There is a deep historical connection between baseball and statistics, from the presence of books like *Moneyball* in pop culture to the personal interest in "sabermetrics" (baseball statistical analysis) that many practicing statisticians hold. For example, Nate Silver, editor-in-chief of political polling site FiveThirtyEight, got his start by building player projection models at Baseball Prospectus, a baseball analytics website (MIT Communication Forum, 2016), while Andrew Gelman of Columbia University frequently blogs about sabermetrics (Gelman, 2011).

What explains this statistical interest in baseball? An important factor here may be that baseball possesses properties that make it uniquely well-suited for analysis. The two most important properties are:

- **Event Independence:** Events ("plate appearances", or every time a hitter faces a pitcher) are mostly independent; a batter's chances of success in one plate appearance does not affect their chances of success in other plate appearances. This means that over the course of a full season (162 games, about 600

plate appearances for a healthy, regular player), a batter’s average outcomes can be treated as a function of an approximately stationary latent batting skill and some noise.

- **Player Independence:** Even more significantly, the performance of two players on the same team can be treated as if they are independent of each other. This makes it both much easier to isolate a player’s skill compared to other team sports like soccer, where it is comparatively difficult to figure out how “skilled” an individual player is because success is a team-level outcome that cannot be easily attributed to the individuals that make up that team.

These nice properties make it much easier both to isolate individual player talent and to aggregate player talent into an accurate measure of team talent. Player and team future performance projections in particular — a natural field of interest for MLB teams, hobbyists, and gambling addicts alike — have proliferated over the past two decades.

1.2 Motivation

I focus my analysis on team hitting performance projection; projecting overall team performance is a more challenging task, and is out of scope for this project. Modern public-facing team projection systems, constructed by sabermetrics websites like Fangraphs and BaseballProspectus, make full use of the two independence assumptions outlined above, and employ broadly similar strategies to generate their projections (Judge, 2025; Szymborski, 2026):

1. Project individual player performance on a rate basis for each player that begins the year on an MLB team. This projection may or may not come with a distribution of potential player outcomes (Szymborski, 2026).
2. For each player, separately project how much playing time the player will receive over the course of the season. This includes two factors: whether the team plays the player every day and for how many games the player is expected to be injured for (Judge, 2025; Szymborski, 2026).
3. Take a weighted average of the “quality” of individual performances (1) by the projected “quantity” of performance (2) to get an overall team performance projection.

The structure above makes the crucial assumption that while playing time may be dependent on a player’s average projected performance, a player’s playing time remains fixed regardless of whether they perform better or worse than expectations. Intuitively, one would think that teams generally give players that perform better than expected more playing time, and players that perform worse less. Exploratory analysis shown in Figure 1 suggests that players that perform better compared to their previous season — a natural rough proxy for “team performance expectations” — tend to also receive more playing time.

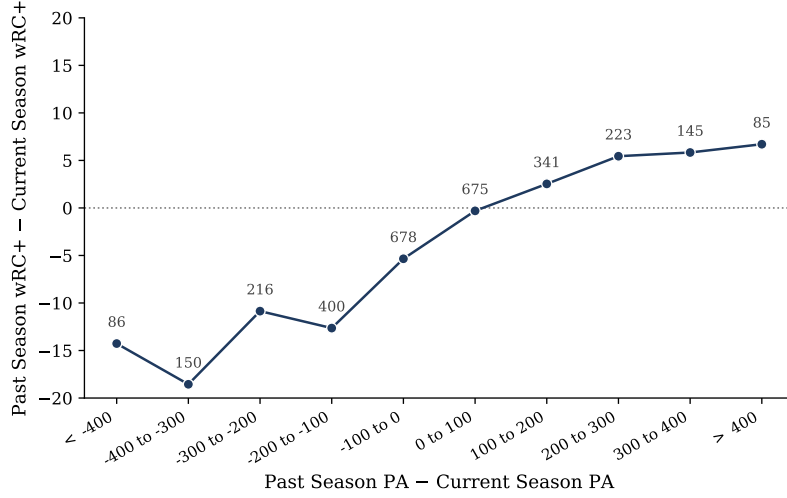


Figure 1: Change in performance by change in playing time

The gains from correctly modeling this dependence are potentially significant; it is commonly acknowledged in the sabermetrics community that it is very difficult to build a player performance projection system that is meaningful better than other models (Tango et al., 2007; Tango, 2012). Instead, most of the difference in projection system quality comes from how playing time projections are implemented (Judge, 2025). As Figure 1 shows, a mere 10% (-10 wRC+) decrease in performance is associated with a decrease of 100-200 PA, which is a 15-30% reduction in playing time.

One of the main goals of this project is to test whether relaxing this independence assumption and building a projection system that allows playing time to be dependent on a player’s realized performance improves team level projections. This approach takes advantage of the comparative advantages of Bayesian machine learning models: natural uncertainty quantification and a posterior predictive distribution to draw from for simulation. Under my proposed approach, players that perform better than expected would receive more playing time, and players that under-perform would receive less.

2 Model Pipeline

2.1 Data and Key Variables

I use data on individual and team MLB season statistics from 2016-2025 from BaseballReference (Sports Reference LLC) and Fangraphs (FanGraphs, a). I use the ‘pybaseball’ Python package to pull data from these websites (LeDoux, 2019). I trained my models on the 2016-2023 data, validated my models by testing on the 2024 data, then train my final selected models on a 2025 season test set.

I use the PyMC Python package (Abril-Pla et al., 2023) for fitting all models trained on my test set.

The main outcome of interest for my individual and team performance projections is wRC+. wRC+ measures individual batting performance relative to all other players in the league and is normalized to 100. For example, a player with a 110 wRC+ is a 10% more productive batter than league average.

The main outcome of interest for my playing time projection is player role. This is a variable that I construct from the individual season statistics and depends on the games started (GS), games (G), and plate appearances (PA) variables. If $\frac{GS}{G} \geq 0.92$ and $PA \geq 50$, the player is classified as a "starter"; if $\frac{0.96 \cdot GS}{G} \geq 0.7$ and $PA \geq 50$, they are "platoon" (part-time); they are classified as a "bench" player otherwise. Each role maps to a set number of plate appearances: starter $\rightarrow$ 600PA; platoon $\rightarrow$ 310PA; bench $\rightarrow$ 70PA. These are based on the median number of plate appearances for these categories of players. This categorization reflects the fact that playing time is not linear since players are typically allotted lumpy amounts of playing time depending on their role.

The individual season statistics also have both information about player characteristics (such as Age, Team, and Fielding Position) and additional features that give more detailed information about player performance (such as strikeout and walk percentage, home runs, and runs scored).

2.2 Structure

In order to build the team level player projections, two models to predict individual player performance and playing time, respectively, are trained and then fed into a Monte Carlo simulation procedure that uses the individual player projections to create team performance projections. Broadly, there are two stages of setting up this team projection pipeline. In the first stage, two types of models are trained:

- **Performance projection:** A regression model that uses the player's past season(s) performance data and other characteristics to predict the player's target season performance.
- **Playing time classification model:** A classification model that uses the player's past season(s) performance and playing time data, characteristics, and (crucially) *target season* performance in order to classify the player as a starter (lots of playing time), platoon (some playing time), or bench player (little playing time).

The classification model is trained to learn the relationship between current season performance and idiosyncratic player background characteristics on the player's role that season. The fact that the playing time model takes in current season performance data to predict current season playing time might seem strange: aren't there issues with data leakage where a player's playing time is always viewed together with their performance? A crucial motivating assumption for the projection pipeline I build here is that *a player's performance affects their playing time, but playing time does not affect their performance*. A helpful distinction to draw here is that between a player's (noisy) observed performance and their unobserved true

talent, which the player’s observed performance should be centered around. Two of the major reasons why this assumption might not hold are:

- **More playing time → fatigue → lower performance:** Baseball is simply not a sufficiently physically taxing enough sport for this mechanism to hold for hitters. This mechanism is very unlikely to be significant.
- **More playing time → learning → higher performance:** While it is true that young, inexperienced players generally improve with time, it is not clear that this is because of getting more playing time or whether this is because of age effects and professional coaching and practice. This mechanism is the more plausible potential violation of my motivating assumption. However, even if this has some minor effect, rookies (players in their first season), who would be most affected by this mechanism, are handled in a slightly different way in my projection pipeline because of data limitations anyway.

In the simulation stage, the regression model is used to predict a distribution of player performance outcomes for a test season (in my case, the 2025 MLB season). I then draw a random outcome for each player from this distribution, feed the outcome into my classification model to assign each player’s a role with associated playing time, normalize the team’s playing time to sum to a fixed amount of team playing time, and compute overall team performance as a weighted average of individual performance and playing time. I then run this process many times to get a distribution of predicted team performance outcomes.

2.3 Performance Projection

The central inferential problem faced when trying to predict future performance from the individual season data I have is that all observed statistics in baseball, especially $wRC+$, are a function of some latent, "true" player skill and some observational noise. The amount of observational noise generally varies with the number of plate appearances (events) that players log in a season:

$$wRC+_{obs} = wRC+_{true} \pm \frac{\sigma_{obs}}{\sqrt{PA}}$$

Additionally, each statistic "stabilizes" at a different number of plate appearances (Alexander, 2026; FanGraphs, b); some statistics, such as strikeout and walk rate, stabilize quickly and have high year-over-year correlation, while others like BABIP and $wRC+$ are fairly noisy and can be fairly different year-over-year. It is desirable to use many of these statistics from past seasons to predict future season performance, but since players have large discrepancies in plate appearances between each other in a season, there might be a large discrepancy in how much weight each player’s observations should receive in a regression. Finally, player skill is non-stationary year-over-year due to age, skill or team changes, and other irreducible random fluctuations.¹ I train three types of Bayesian linear regression (BLR) models with these inferential problem in mind:

¹Additionally, I do not have past season data on rookies because it is their first season in the

1. **BLR on past performance:** As a baseline, I train a Bayesian linear regression model that uses only a player’s previous season wRC+ (minimum 100PA) to predict their future wRC+. I also train a model that uses the player’s previous season wRC+ and the change in wRC+ from two seasons before to a season before, which attempts to capture a signal of the player’s skill evolution. I also trained models that use age as an additional predictor but discarded these because of performance on the validation set.
2. **BLR on key performance statistics:** To circumvent the problem with using noisy past season statistics, I built a number of BLR models using only statistics that have been shown in past sabermetric analysis to stabilize with a small number of plate appearances (100PA) while also being stable year-over-year and predictive of current season wRC+. Candidate statistics included Barrel%, K%, BB%, K-BB%, Hard%, and more. I also considered training a Bayesian LASSO regression model using a number of these indicators and interactions, but I ultimately did not think this would be promising based on the performance of the component models I trained in this stage and the relative lack of training data. Based on model evaluation, I select a model that uses only Barrel% and K-BB% as predictors.
3. **Bayesian MARCEL:** This primary inspiration for this model is the classical MARCEL projection model. As its creator, Tom Tango, has said, it is “the most basic forecasting system you can have” (Tango, 2012), and it serves as a competitive baseline projection system that is surprisingly difficult to beat. The model weights the past three years of performance, applies regression to the league mean performance (with stronger regression for players with fewer plate appearances), and adjusts the performance estimate according to an aging curve. The original model uses pre-determined, deterministic weights to create performance projections. I base my model on work done by PyMC Labs to build a Bayesian version of MARCEL to predict hard hit rates in baseball (Fonnesbeck, 2024). In their work, they fit a hierarchical model with Markov Chain Monte Carlo (MCMC) and let the data determine what weights are used rather than MARCEL’s fixed ones. I adapt their methodology to fit a Bayesian MARCEL model for wRC+ using the past two seasons of player data. The Bayesian methodology has the additional crucial advantage of giving me a predictive distribution of outcomes to draw from for my simulation as opposed to a deterministic performance projection.

In addition to these models, I include MARCEL as a benchmark projection. In Table 1, I compare the performance of the models on how accurate they are as measured by Root Mean Squared Error. For my models, I also report the percentage of wRC+ observations are within the 94% credible interval constructed by my models.

MLB. I handle this problem by assigning a fixed distribution of performance outcomes to rookies based on the empirical distribution of rookie performance from 2017-2023. This is consistent with how other projection models like MARCEL (Tango, 2012) deal with rookies. For the playing time projection, I similarly treat rookies differently and fit a separate simple logistic regression for rookie playing time. Handling rookie projections has been a common hurdle for other projection systems.

Table 1: Model performance on 2025 test set

Model	RMSE	94% Coverage
MARCEL	26.258	—
Bayesian MARCEL	24.604	91.4%
BLR: wRC+	26.488	92.4%
BLR: wRC+, wRC+ delta	25.683	92.5%
BLR: Barrel%, K-BB%	28.070	93.6%

All trained models have good coverage of the 94% interval. Of my models, Bayesian MARCEL performs the best based on RMSE and notably outperforms the MARCEL baseline, while the component projection model using Barrel% and K-BB% performs worse than the baseline. The other two models exhibit similar performance to MARCEL.

2.4 Playing Time Projection

For the playing time projection, I use Bayesian ordered multinomial logistic regression to classify players as "starter", "platoon" (part-time), or "bench" players. The primary challenge with building the playing time projection model was not the model building itself but finding player classification rules in the data pre-processing phase that would correctly assign players to their intended role. The starter/platoon/bench split is motivated by substantive knowledge of baseball, but there is a challenge with assigning players to these roles since these roles are never officially confirmed. Additionally, a single mapping from plate appearances and games started to player role and from player role to playing time is severely obfuscated by player injuries that are not recorded in the data.

I tested a number of models, all of which also include current season wRC+ because of the reasons outlined in Section 2.2, that include past season player performance metrics and characteristics. The features for the two models I settle on training are:

1. **Model 1:** current season wRC+, whether the player was a starter the previous year, and number of plate appearances the previous year
2. **Model 2:** current and past season wRC+, whether the player was a starter, baserunning skill the previous year, defensive value the previous year, whether the player was a starter on the same team the previous year

The second model attempts to account for past year performance factors, especially non-batting factors, that might affect playing time.

Unlike with the performance projections models, there is no natural external benchmark for this model. A big reason why there is no natural comparison for my playing time model is that my playing time model trains on actual current season wRC+ while other models generally do not use current season data. Furthermore,

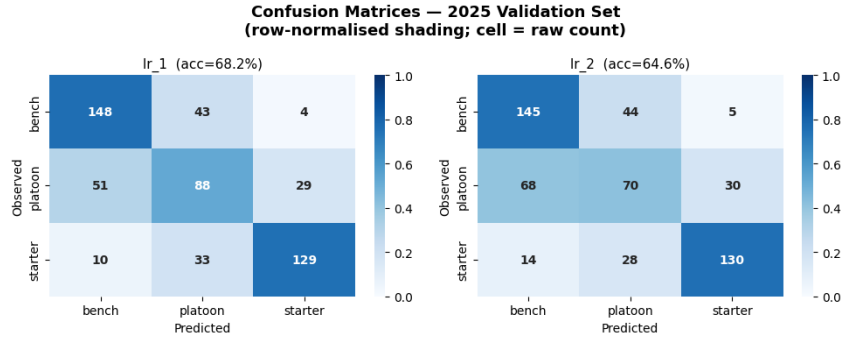


Figure 2

some projection websites do their playing time projections ”by hand”, meaning that they do not use an algorithmic process for their projections (Szymborski, 2026). Figure 2 gives information on how well the two models succeed at correctly classifying player roles. Model 1, the simpler model, does a better job of correctly classifying players.

2.5 Simulation

After the regression and classification models are trained, I run Monte Carlo simulations to generate a distribution of team level performance projections using the following procedure:

1. For each player on the team:
 - (a) Draw wRC+ from the performance projection model’s predictive distribution for the player.
 - (b) Feed this wRC+, in addition to the player’s past season performance, playing time, and characteristics, into the playing time classification model to predict the player’s role (starter, platoon, bench) for the season.
 - (c) The playing time classification model assigns probabilities that the player, given their performance, is assigned each role. Use these probabilities to draw an assigned role for the player. Assign the player playing time (plate appearances) based on the role they were assigned.
2. Each team has a fixed ”budget” of total playing time they can give to players in a season. This ”budget” is empirically about 6100 plate appearances per team, with little variation between teams [CITE: Fangraphs Major League Leaderboards - 2024 - Batting page]. Aggregate projected playing time for each player on the team and normalize each player’s playing time so that the sum of projected plate appearances for each team equals 6100 plate appearances.
3. Take an average of each player’s projected performance from step 1(a) weighted by their projected plate appearances from step 2 to obtain the team’s overall wRC+ for the season.

4. **Monte Carlo:** Repeat steps 1-3 many times to obtain a distribution of projected overall team wRC+.

Monte Carlo simulations are also used by other prominent baseball projection systems to generate team performance projections (Szymborski, 2026). I run this procedure to generate team performance distributions for every combination of regression and classification model that I train. This means that I produce team projections for 8 different model combinations.

3 Evaluation

The first criteria I am interested in is whether my hypothesis that modeling the dependency of playing time on player performance can improve player projection models holds. This was the question that initially motivated my project.

To test this hypothesis, I create a "deterministic" version of my 8 model pipelines. This means that for each player on a team, I take their mean performance projection from the regression model and plug this into the classification model to get a role, and playing time, projection. Like in the simulation, I then normalize playing time across a team to 6100 and compute team wRC+ to get a single point estimate of team performance. To get a sense of whether my hypothesis seems promising, I compare the RMSE of these deterministic projections to the mean team performance projections generated by my simulations. In Table 2, the column Det. RMSE gives the RMSE of the deterministic version of the model combination.

Table 2: Simulation model performance comparison

Reg. Model	Cls. Model	RMSE	Det. RMSE	80% Coverage
Bayesian MARCEL	Model 1	7.925	7.650	69.0%
BLR: wRC+	Model 1	7.977	8.116	79.3%
BLR: wRC+	Model 2	8.023	7.814	72.4%
Bayesian MARCEL	Model 2	8.050	7.795	69.0%
BLR: wRC+, wRC+ delta	Model 1	8.403	8.245	62.1%
BLR: wRC+, wRC+ delta	Model 2	8.502	8.136	58.6%
BLR: Barrel%, K-BB%	Model 1	9.645	9.828	65.5%
BLR: Barrel%, K-BB%	Model 2	9.654	9.823	65.5%

The different model combinations generally undercover at the 80% credible interval level, but coverage is generally acceptable, especially for the best performing models by RMSE. One note is that coverage might not be a super precise measure here because there are only 30 teams in the sample. Also encouraging is the fact that the best performing regression and classification models (Bayesian MARCEL and Model 1) combine to produce the best simulation model by RMSE. However, there is no significant gap in performance in terms of RMSE when the best performing simulation models are compared to their deterministic counterparts. There is also no clear relationship between absolute simulation model performance and

relative performance of the simulation versus the deterministic models, so it is unclear if these results would be different for better performing simulation models. Another interesting question is how important the relative quality of the regression and classification models are, and what the nature of the interaction between the two is.

The second criteria I am interested in is how my projections compare to team performance projections made by other sabermetrics websites. Because of some limitations in the data that my classification model uses and the fact that I do not have a separate injury model, my expectations were that my projections would be outperformed by the industry leading models. I was pleasantly surprised to find that the performance of my models for projecting 2025 team hitting performance was right in line with a selection of the best industry models (Table 3).

Table 3: 2025 Industry projection system performance

System	RMSE
ZiPS	7.562
Steamer	7.608
ATC	7.719
THE_BAT	7.924
OOPSY	8.154

Overall, the idea of relaxing the independence assumption between player performance and playing time that motivated my project was not borne out in my results; however, the performance of my projection pipeline overall was close to other models developed by sabermetricians. Although the accuracy of these systems was compared over a small test set of 30 teams, the results might indicate that the overall model pipeline that I built holds promise.

In particular, I believe that my Bayesian MARCEL regression model is a promising model architecture for performance projection, and that the natural uncertainty quantification it gives is a meaningful advantage. In terms of my classification models, I believe there is a lot of room to improve. Aside from properly modeling differential injury risk among players, I think sequential models would be interesting to explore. This is because the implicit assumption in my classification model is that teams get to see the player’s whole-season performance before allocating whole-season playing time. This is a slightly unrealistic abstraction, and it would be more accurate to build a model that account for the fact that players actually have their role and playing time adjusted throughout the season as teams get more performance data on the players.

Code

I have uploaded the Jupyter notebook that trains, simulates, and evaluates the models I use to produce the final results in this project. However, my project also required a large number of auxiliary code files for extensive exploratory data analysis, candidate model fitting and selection, and especially data scraping and pre-processing. For this reason, I am also including a link to my GitHub project subfolder. The rest of my code files can be found here.

References

- J. Mizels, B. Erickson, and P. Chalmers, “Current state of data and analytics research in baseball,” *Current Reviews in Musculoskeletal Medicine*, vol. 15, no. 6, pp. 510–520, 2022.
- MIT Communication Forum, “A conversation with Nate Silver,” 2016, mIT Communication Forum, published on Medium. Accessed 2026. [Online]. Available: <https://commforum.mit.edu/a-conversation-with-nate-silver-3cab28b02806>
- A. Gelman, “Baseball ProQUESTus: A statistician rereads Bill James,” Baseball Prospectus, 2011, published 5 May 2011. Accessed 2026. [Online]. Available: <https://www.baseballprospectus.com/news/article/13810/baseball-proquestus-a-statistician-rereads-bill-james/>
- J. Judge, “2024 projection review: Batter playing time,” RotoGraphs Fantasy Baseball, FanGraphs, 2025, accessed 2026. [Online]. Available: <https://fantasy.fangraphs.com/2024-projection-review-batter-playing-time/>
- D. Szymborski, “The late-January ZiPS projected standings update,” FanGraphs Baseball, 2026, accessed 2026. [Online]. Available: <https://blogs.fangraphs.com/the-late-january-zips-projected-standings-update/>
- T. Tango, M. Lichtman, and A. Dolphin, *The Book: Playing the Percentages in Baseball*. Potomac Books, 2007.
- T. Tango, “Marcel the monkey forecasting system,” 2012, accessed 2026. [Online]. Available: <http://www.tangotiger.net/marcel/>
- Sports Reference LLC, “Baseball reference,” Sports Reference LLC, accessed 2026. [Online]. Available: <https://www.baseball-reference.com>
- FanGraphs, “FanGraphs baseball,” FanGraphs, accessed 2026. [Online]. Available: <https://www.fangraphs.com>
- J. LeDoux, “pybaseball: Python package for baseball data,” 2019, gitHub repository. Accessed 2026. [Online]. Available: <https://github.com/jldbc/pybaseball>

- O. Abril-Pla, V. Andreani, C. Carroll, L. Dong, C. J. Fonnesbeck, M. Kochurov, R. Kumar, J. Lao, C. C. Luhmann, O. A. Martin, M. Osthege, R. Vieira, T. Wiecki, and R. Zinkov, “PyMC: A modern and comprehensive probabilistic programming framework in Python,” *PeerJ Computer Science*, vol. 9, p. e1516, 2023. [Online]. Available: <https://doi.org/10.7717/peerj-cs.1516>
- Z. Alexander, “A guide to statistical stabilization and regression,” Birdland Metrics, 2026, accessed May 2026. [Online]. Available: <https://birdlandmetrics.com/articles/>
- FanGraphs, “Sample size,” FanGraphs Sabermetrics Library, accessed 2026. [Online]. Available: <https://library.fangraphs.com/principles/sample-size/>
- C. Fonnesbeck, “Bayesian MARCEL: A simple, probabilistic model for MLB player projections,” PyMC Labs Blog, 2024, accessed 2026. [Online]. Available: <https://www.pymc.io/blog/bayesian-marcel>